Getting Started Guide

Configure Ping Access Management (PingAM) / ForgeRock Access Management with LexisNexis Dynamic Decision Platform (DDP) Nodes

Version 2.0.2

October 2026

LexisNexis® RISK SOLUTIONS

Table of Contents

SCOPE 3
Document Organization 3
Intended Audience 3
LEXISNEXIS DYNAMIC DECISION PLATFORM NODE OVERVIEW 4
LexisNexis DDP Profiler Node 4
Input 4
Configuration 4
Output 5
Callbacks 5
Outcomes 5
LexisNexis DDP Query Node 5
Input 6
Configuration 6
Output 7
Callbacks 7
Outcomes 7
LexisNexis DDP Review Status Node 7
Input 7
Configuration 7
Output 8
Callbacks 8
Outcomes 8
LexisNexis DDP Reason Code Node 8
Input 8
Configuration 9
Output 9
Callbacks 9
Outcomes 9
LexisNexis DDP Update Review Node 10
Input 10
Configuration 10
Output 10
Callbacks 11
Outcomes 11
NODES INSTALLATION 12
DYNAMIC DECISION PLATFORM PORTAL CONFIGURATION 13
Retrieve OrgID and API Key 13
Risk Assessment Policy 14
AUTHENTICATION TREE CONFIGURATION 16
Authentication Tree: Login-Risk-Assessment 16
Authentication Tree: LNRS-StepUp-Update 22

SCOPE

LexisNexis Risk Solutions (LNRS) Dynamic Decision Platform (DDP) unifies the key components of fraud prevention into a single, holistic risk decision platform. This document contains the detailed steps and supporting information required to install and configure Ping Access Management (PingAM), formerly ForgeRock Access Management (AM) with LexisNexis DDP Authentication Nodes. This guide is intended to install a simple configuration to support testing.

The following are architecture assumptions and limitations:

- PingAM / ForgeRock Access Management server has been previously installed
- PingAM / ForgeRock Access Management server has an Identity Store configured
- A test account has been configured via Identities that includes first name, last name, email address, and login username

Document Organization

This document is divided into four sections as follows:

- **Scope.** Defines the purpose of this document.
- **LexisNexis DDP Nodes**. Provides an overview of the nodes available for authentication tree integration.
- **LexisNexis Nodes Installation.** Provides information to install the LexisNexis DDP nodes upon an on-premise PingAM / ForgeRock Access Management server.
- **LexisNexis DDP Portal.** Provides detailed information regarding the configuration of the Dynamic Decision Platform (DDP) to include configuration of a simple policy and how to access configuration parameters required for the authentication tree.
- **Authentication Tree Configuration.** Provides detailed steps to configure an authentication tree with LexisNexis DDP for an authentication/login use case combined with risk assessment to detect suspicious activity associated to the login event.

Intended Audience

Table 1 contains a list of the different readers to whom this documentation is directed and how they will use this document.

Table 1: Intended Audience and Reading Suggestions

Reader Type	Use of this document
Developers	This guide is intended for software developers to be able to install and configure PingAM / ForgeRock for testing and integration.
Administrators	This guide is also intended for administrator to be able to install and configure PingAM / ForgeRock for testing and integration.

LEXISNEXIS DYNAMIC DECISION PLATFORM NODE OVERVIEW

LexisNexis® Dynamic Decision Platform (DDP) unifies the key components of fraud prevention into a single, holistic risk decision platform. DDP connects multiple data sources and intelligence signals, enabling real-time orchestration of risk decisions. By integrating multi-signal insights, custom attributes and AI-enabled models, the platform delivers faster, more accurate outcomes while reducing operational complexity.

LexisNexis Dynamic Decision Platform (DDP) nodes are available for both PingOne Advanced Identity Cloud (PingOne AIC), formerly ForgeRock Identity Cloud, and Ping Access Management (PingAM), formerly ForgeRock Access Management. The nodes provide production ready connectivity and orchestration of all LexisNexis products to include ThreatMetrix, BehavioSec, Emailage, PhoneFinder, InstantID, FlexID and more. The nodes also support a wide variety of use cases to include login, password change, account creation, and payment to name a few. When the nodes are integrated, the risk assessment policy hosted within the LexisNexis DDP cloud returns a risk score mapped to defined outcomes such as step-up authentication, passing without further friction, or rejecting resulting in blocking the transaction.

To include LexisNexis DDP nodes in an authentication tree, enter LexisNexis into the Filter nodes to get a listing of available capabilities. For risk assessment, the following nodes are available:

- LexisNexis DDP Profiler
- LexisNexis DDP Query
- LexisNexis DDP Review Status
- LexisNexis DDP Reason Code
- LexisNexis DDP Update Status

LexisNexis DDP Profiler Node

This node will integrate the device intelligence and fingerprinting JavaScript Tags onto a Ping/ForgeRock Page Node. This is typically placed onto a Login Page, Payment Page, or Account Creation page as part of a risk assessment use case.

Input

The LexisNexis DDP Profiler node has no required inputs.

Configuration

The LexisNexis DDP Profiler node has the following configuration parameters:

- **Org ID** - Org ID is the unique id associated with DDP generated for your organization.
- **Page ID** - The Page ID is an identifier to be used if you place the profiling on multiple page nodes. This is an optional parameter and can be left blank.
- **Profiler URI** - DDP Profiler URI. This can be the Basic Profiling URL or the Enhanced Profiling vis Hosted SSL URL. The default configuration is the Basic Profiling URL for the global region.

- **Use Client Generated Session IDs** - If DDP JavaScript Tags have been separately integrated onto an customer hosted webpage or mobile device, this configuration allows for sending the unique Session ID to Ping / ForgeRock through `HiddenValueCallback` as part of an API Request.
- **Tag Location** – When integrating using PingAM or PingAIC hosted HTML pages, typically through a Page Node, this configuration defines the tag placement. The HEAD option is optimal for PingAM deployments, whereas the iFrame option is required for PingAIC deployments.

Output

The LexisNexis DDP Profiler node has the following outputs placed into shared state:

- **Session ID** - The unique Session ID value used with JavaScript Tags will be contained in the state variable named `ddp.session_id`. This state variable is further used by the LexisNexis DDP Query node to invoke DDP Session Query APIs. The Profiler is only needed for DDP ThreatMetrix and BehavioSec products that combine the information collected by JavaScript and identified by the Session ID.
- **Failure Reason** - If an error outcome is generated, the state variable `ddp.failure_reason` will contain text associated with the error condition.

Callbacks

The LexisNexis DDP Profiler node has the following callbacks:

- When integrating with a client application that performs the device profiling, the Ping/ForgeRock platform is integrated via API. The Ping/ForgeRock API that invokes the LexisNexis DDP Profiler node will get the following callback:
    - `HiddenValueCallback` - Container to send Session ID to Ping/ForgeRock when integrating a client application via APIs

Outcomes

The LexisNexis DDP Profiler node has the following outcomes:

- **Next** - This outcome is triggered when profiling has successfully completed
- **Error** - This outcome is triggered when there is a fundamental integration error. First attempt to fix the integration error by looking at debug log files for the node to determine if the integration error is due to configuration. If the configuration looks accurate, then open a support case with LexisNexis Risk Solutions.

LexisNexis DDP Query Node

This node makes a request LexisNexis DDP API Request to either: (i) Session Query API, or (ii) Attribute Query API. The main difference is that Session Query API requires the LexisNexis DDP Profiler node to perform device intelligence, whereas the Attribute Query does not involve device intelligence. Attribute query is helpful in situations where a LexisNexis Risk Solutions product such as Emailage or InstantID can be invoked for a risk assessment without any device intelligence.

Input

The LexisNexis DDP Query node retrieves the following from the journey shared state based upon the configuration of the Attribute Source parameter.

- **AM Username (`username`)** - Required in shared state, or `objectAttributes`, to resolve the desired AM identity when the **Attribute Source = User Directory**.
    - When **Query Attributes** are defined and the **Add Attributes to API Request** is enabled, the node will fetch user attributes based on the attribute name configured from the AM Identity of the corresponding `username`.
- **objectAttributes** - Required when the **Attribute Source = Shared State**.
    - When **Query Attributes** are defined and the **Add Attributes to API Request** is enabled, the node will fetch user attributes from shared state.

Configuration

The LexisNexis DDP Query node has the following configuration parameters:

- **Org ID** - Org ID is the unique id associated with DDP generated for your organization.
- **API Key** - This is the unique API key generated by DDP associated to the Org ID.
- **Policy** - The policy to be used for the query. The policy is configured in the DDP Portal which is a set of decision policy rules that together generate the risk assessment policy score and outcome.
- **Base URL** - Defines the domain URL for the DDP region where API Requests are to be sent. The default value is the global region.
- **Service Type** - Defines the API Response output fields returned from the API Request. The default configuration is session-policy. See the DDP Knowledge Base (KB) for a full list of service types.
- **Event Type** - Specifies the type of transaction or event. The default configuration is login. See the DDP KB for a full list of event types.
- **Query Type** - Defines the query type to send as the DDP API Request. Session Query requires device intelligence to be collected previously by the LexisNexis DDP Profiler node and Attribute Query does not require device profile information.
- **Add Attributes to API Request** - If you'd like to add additional parameters to the DDP API Request, enable this option. In general, it is preferred to add as much data as possible to the API Requests as this will improve the fidelity of the risk assessment.
- **Attribute Source** - Defines where additional attributes (if configured) are to be fetched at runtime. This is a dropdown list that contains the options User Directory and Shared State. User Directory will look for attributes in the Identity Store, and Shared State looks in the shared memory of the authentication tree/journey.
- **Query Attributes** - This is a list of DDP attributes (e.g. "key") to authentication journey attributes (e.g. "value"). The Attribute Source configuration defines where the values will be fetched. If the values cannot be fetched, the API Request will not include the DDP attribute.

Output

The LexisNexis DDP Query node has the following outputs placed into shared state:

- **Session ID** - The unique Session ID value used with JavaScript Tags contained in the state variable named `ddp.session_id` is removed following the success of this node. The value is no longer needed for integration.
- **DDP Query API Response** - The full API Response is contained in the state variable named `ddp.query_api_response`. This value is further processed by the LexisNexis DDP Review Status node or the LexisNexis DDP Reason Code node.
- **Failure Reason** - If an error outcome is generated, the state variable `ddp.failure_reason` will contain text associated with the error condition.

Callbacks

The LexisNexis DDP Query node does not have any callbacks as there is no user interface displayed.

Outcomes

The LexisNexis DDP Query node has the following outcomes:

- **Next** - This outcome is triggered when the DDP Query API has successfully completed
- **Error** - This outcome is triggered when (i) the API results in non-success, or (ii) there is a fundamental integration error. First attempt to fix the integration error by looking at debug log files for the node to determine if the integration error is due to configuration. If the configuration looks accurate, then open a support case with LexisNexis Risk Solutions.

LexisNexis DDP Review Status Node

This node analyzes the response from the LexisNexis DDP Query node and routes based on the API Response `review_status`. The possible outcomes to route are `Pass`, `Challenge`, `Review` or `Reject` node outcomes. If an unknown session occurred as a result of profiling and the DDP API Response contains the unknown session condition, the LexisNexis DDP Review Status Node will follow the configured Unknown Session Action.

Input

The LexisNexis DDP Review Status node retrieves the following from the journey shared state:

- **DDP Query API Response** - The full API Response is contained in the state variable named `ddp.query_api_response`.

Configuration

The LexisNexis DDP Review Status node has the following configuration parameters:

- **Unknown Session Action** - If an “unknown session” is encountered at runtime, this allows the system administrator to define the behavior in the unlikely event this occurs at runtime. Unknown sessions occur for a variety of reasons where device profiling has failed.
- **Store DDP Query API Response** - If enabled, the **DDP Query API Response** will remain in shared state. If not enabled, then the state variable `ddp.query_api_response` will be removed from shared state.

Output

The LexisNexis DDP Review Status node has the following outputs placed into shared state:

- **DDP Query API Response** - If the **Store DDP Query API Response** is enabled, then the full LexisNexis Query API Response contained in state variable `ddp.query_api_response` will remain in shared state. Otherwise, state variable `ddp.query_api_response` will be removed from shared state.
- **DDP Reason Codes** - The reason codes from the DDP Policy will be placed into state variable `ddp.query_reason_codes` for all outcomes.
- **DDP Request ID** - The Request ID (e.g. attribute `request_id`) associated to the DDP Query API will be placed into state variable `ddp.request_id`. This value is used by the LexisNexis Update node to add truth data to a risk event, as well as this can be used to link other LexisNexis Risk Solutions product APIs as an identifier.
- **Failure Reason** - If an error outcome is generated, the state variable `ddp.failure_reason` will contain text associated with the error condition.

Callbacks

The LexisNexis DDP Review Status node does not have any callbacks as there is no user interface displayed.

Outcomes

The LexisNexis DDP Review Status node has the following outcomes:

- **Pass** - This outcome is triggered when the DDP Query API Response `review_status` is equal to pass.
- **Review** - This outcome is triggered when the DDP Query API Response `review_status` is equal to review.
- **Challenge** - This outcome is triggered when the DDP Query API Response `review_status` is equal to challenge.
- **Reject** - This outcome is triggered when the DDP Query API Response `review_status` is equal to reject.
- **Error** - This outcome is triggered when there is a fundamental integration error. First attempt to fix the integration error by looking at debug log files for the node to determine if the integration error is due to configuration. If the configuration looks accurate, then open a support case with LexisNexis Risk Solutions.

LexisNexis DDP Reason Code Node

This LexisNexis DDP Reason Code node analyzes the response from the LexisNexis DDP Query node and routes based on the API Response `reason_code`. Reason code outcomes are required to be configured so that appropriate outcome routing can occur at runtime. The reason codes correspond to the DDP Portal policy configuration for possible outcomes. Reason codes are generally utilized when the four (4) default outcomes for review status are not sufficient for branching in the Ping/ForgeRock authentication tree/journey.

Input

The LexisNexis DDP Reason Code node retrieves the following from the journey shared state:

- **DDP Query API Response** - The full API Response is contained in the state variable named `ddp.query_api_response`.

Configuration

The LexisNexis DDP Reason Code node has the following configuration parameters:

- **Reason Code Outcomes** - A list of Reason Codes that to check from a DDP Query API Response. When a Reason Code is added to this list, a new outcome will display on the node outcomes. The node will iterate through the configured Reason Code list until a Reason code match is found and will return that outcome. Otherwise, the None Triggered outcome will be returned. Reason Code outcomes are case sensitive and must match the DDP Portal policy.
- **Unknown Session Action** - If an "unknown session" is encountered at runtime, this allows the system administrator to define the behavior in the unlikely event this occurs at runtime. Unknown sessions occur for a variety of reasons where device profiling has failed.
- **Store DDP Query API Response** - If enabled, the **DDP Query API Response** will remain in shared state. If not enabled, then the state variable `ddp.query_api_response` will be removed from shared state.

Output

The LexisNexis DDP Reason Code node has the following outputs placed into shared state:

- **DDP Query API Response** - If the **Store DDP Query API Response** is enabled, then the full LexisNexis Query API Response contained in state variable `ddp.query_api_response` will remain in shared state. Otherwise, state variable `ddp.query_api_response` will be removed from shared state.
- **DDP Reason Codes** - The reason codes from the DDP Policy will be placed into state variable `ddp.query_reason_codes` for all outcomes.
- **DDP Request ID** - The Request ID (e.g. attribute `request_id`) associated to the DDP Query API will be placed into state variable `ddp.request_id`. This value is used by the LexisNexis Update node to add truth data to a risk event, as well as this can be used to link other LexisNexis Risk Solutions product APIs as an identifier.
- **Failure Reason** - If an error outcome is generated, the state variable `ddp.failure_reason` will contain text associated with the error condition.

Callbacks

The LexisNexis DDP Reason Code node does not have any callbacks as there is no user interface displayed.

Outcomes

The LexisNexis DDP Reason Code node has the following outcomes:

- Configured outcomes - Using the **Reason Code Outcomes** configuration, a dynamic set of outcomes is defined. The corresponding outcome is triggered when a syntax match is processed in the list of codes within the DDP Query API Response `reason_code` attribute.
- **None Triggered** - This outcome is triggered when the DDP Query API Response `reason_code` does not match any of the configured outcomes.

- **Error** - This outcome is triggered when there is a fundamental integration error. First attempt to fix the integration error by looking at debug log files for the node to determine if the integration error is due to configuration. If the configuration looks accurate, then open a support case with LexisNexis Risk Solutions.

LexisNexis DDP Update Review Node

The LexisNexis DDP Update node associated retrospective truth data to a DDP API event using the Request ID from the DDP Query API Response node. The typical authentication tree/journey will perform a LexisNexis DDP Query and if step-up authentication is involved, the LexisNexis DDP Update node is integrated to provide additional details on the event. Truth data is incredibly beneficial for tuning of a DDP policy and overall fraud detection.

Input

The LexisNexis DDP Update node retrieves the following from the journey shared state:

- **DDP Request ID** - The Request ID (e.g. attribute `request_id`) associated to the DDP Query API will be fetched from state variable `ddp.request_id`. This value is used by the LexisNexis Update node to add truth data to a risk event.

Configuration

The LexisNexis DDP Update Node has the following configuration parameters:

- **Org ID** - Org ID is the unique id associated with DDP generated for your organization.
- **API Key** - This is the unique API key generated by DDP associated to the Org ID.
- **Base URL** - Defines the domain URL for the DDP region where API Requests are to be sent. The default value is the global region.
- **Event Tag** - This represents the event disposition and outcome of the DDP Query API event. Generally, the `challenge_init` is configured prior to sending a Step-Up authentication request in the event the transaction is abandoned. Following a step-up authentication, either `challenge_pass` or `challenge_fail` is sent to DDP.
- **Step-Up Method** - This is the authentication challenge method used within the Ping/ForgeRock authentication tree to report retrospective truth data for the overall transaction.
- **Notes** - An optional notes parameter that allows you to append any notes such as why the review status is being updated.

Output

The LexisNexis DDP Update node has the following outputs placed into shared state:

- **Failure Reason** - If an error outcome is generated, the state variable `ddp.failure_reason` will contain text associated with the error condition.

Callbacks

The LexisNexis DDP Update node does not have any callbacks as there is no user interface displayed.

Outcomes

The LexisNexis DDP Update node has the following outcomes:

- **Next** - This outcome is triggered when the DDP Update API has successfully completed
- **Error** - This outcome is triggered when (i) the API results in non-success, or (ii) there is a fundamental integration error. First attempt to fix the integration error by looking at debug log files for the node to determine if the integration error is due to configuration. If the configuration looks accurate, then open a support case with LexisNexis Risk Solutions.

NODES INSTALLATION

This section describes how to deploy the LexisNexis DDP Authentication Nodes to PingAM / ForgeRock Access Management hosted on Apache Tomcat. The server will need to be stopped and restarted for the nodes to be properly deployed. This instruction assumes an Apache Tomcat application web server.

1. Stop the Tomcat server
2. Remove any previously installed versions of LexisNexis DDP Nodes from the server:

   Directory: `<pingaminstall>/tomcat/webapps/openam/WEB-INF/lib`
3. Copy the LexisNexis DDP Nodes media as follows:

   Filename: `lexisnexis-ddp-2.0.0.jar`

   Directory: `<pingaminstall>/tomcat/webapps/openam/WEB-INF/lib`
4. Restart Tomcat server from command line

The latest release of the LexisNexis DDP Authentication Nodes contains new configuration parameters that differ from the previous version 1.x releases. The new release will handle backwards compatibility for any existing authentication trees, as well as insert default values for new configuration parameters. Be sure to inspect all nodes and configure as desired.

DYNAMIC DECISION PLATFORM PORTAL CONFIGURATION

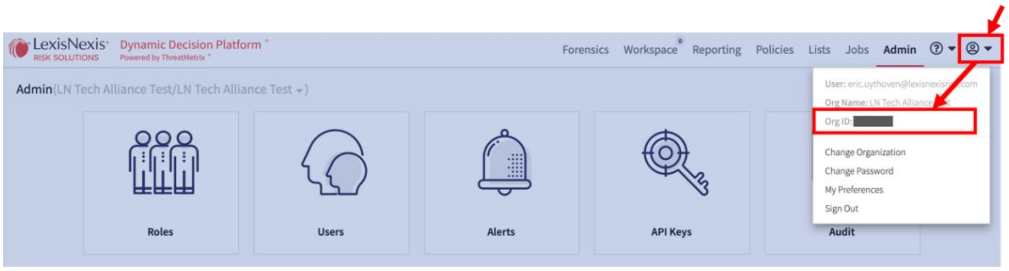
This section defines the high-level LexisNexis Dynamic Decision Platform (DDP) Portal configuration items that will be needed for the overall configuration. There are two main categories of configuration, mainly,

- Organization ID and API Key for the REST API interfaces. This information is needed by the LexisNexis DDP Nodes and will be entered as part of configuration.
- LexisNexis DDP Portal Policies. The configured policy within DDP Portal provides real-time contextual risk assessment. For the purposes of the getting started guide and to have a simple test configuration for different outcomes, the "default" policy will be configured to detect variations in the user browser agent that is used for testing, mainly Chrome, Firefox, Safari, Microsoft Edge and Internet Explorer.

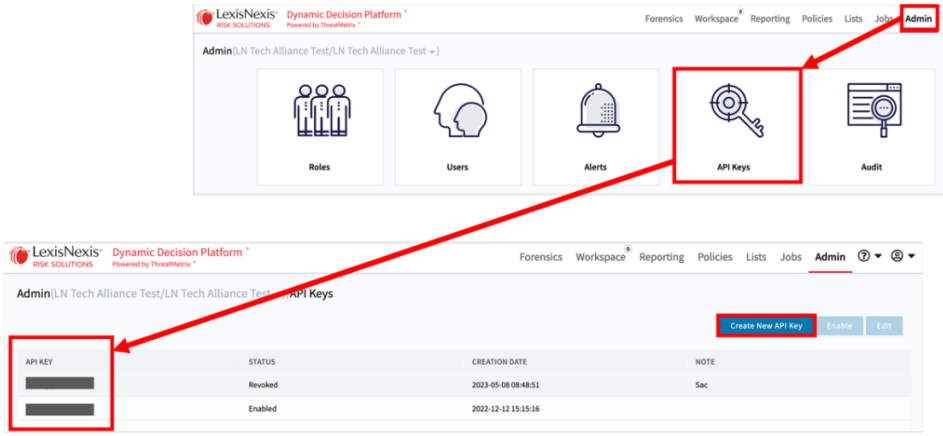
Retrieve OrgID and API Key

To retrieve the DDP Portal values for Organization ID and API Key, perform the following steps.

1. Access **DDP Portal** over the internet by logging into your administrative account with credentials provided by LexisNexis Risk Solutions.
2. From the **DDP Portal** home page, select the user information dropdown that will display username, OrgName and OrgID. This will be the OrgID to enter for the configuration of the LexisNexis DDP Nodes.



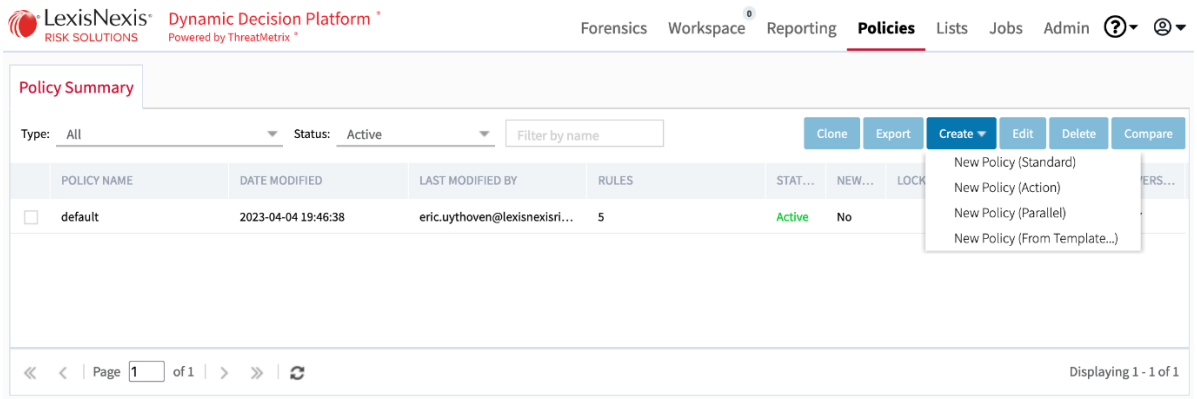
3. Within the **DDP Portal** home page, select **Admin** followed by selecting the **API Keys** tile. Retrieve the value for API Key. In the event no API Key is listed, select the **Create New API Key** button to generate a new key. This will be the API Key to enter into the configuration of the LexisNexis IIDQA Nodes. The API Key is to be protected. Do not email or keep this value in cleartext on any computer system.



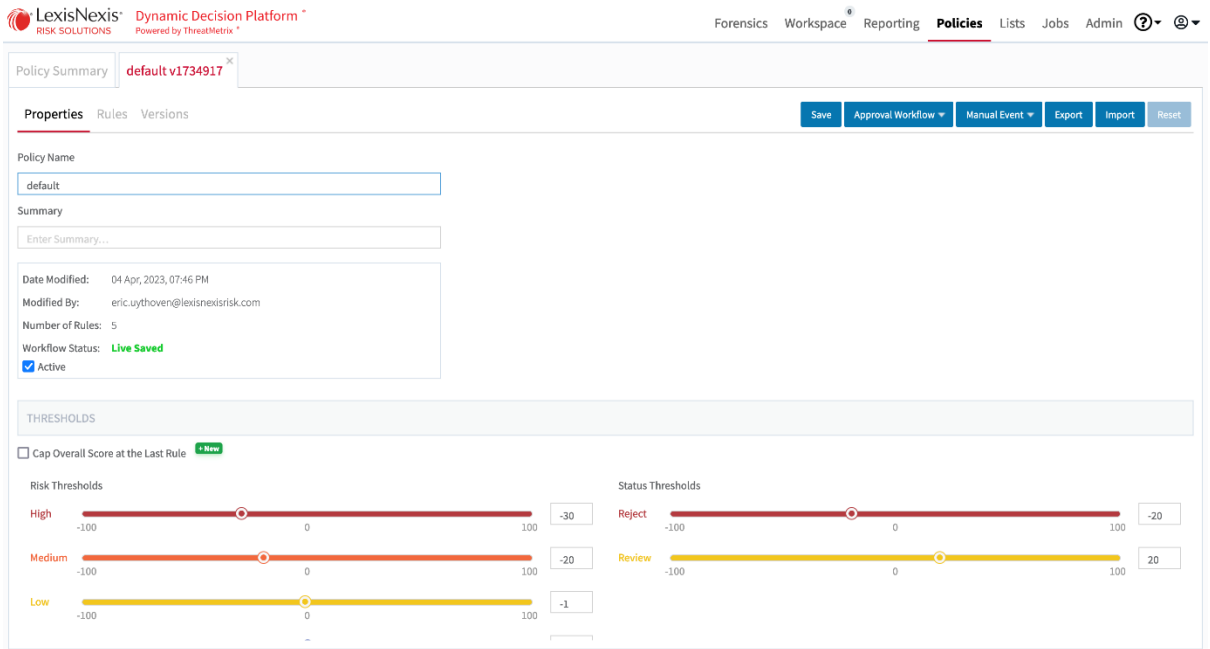
Risk Assessment Policy

For the purposes of the getting started guide and to have a simple test configuration for different outcomes, the "default" policy will be configured to detect variations in the user browser agent that is used for testing, mainly Chrome, Firefox, Safari, Microsoft Edge and Internet Explorer. Perform the following steps.

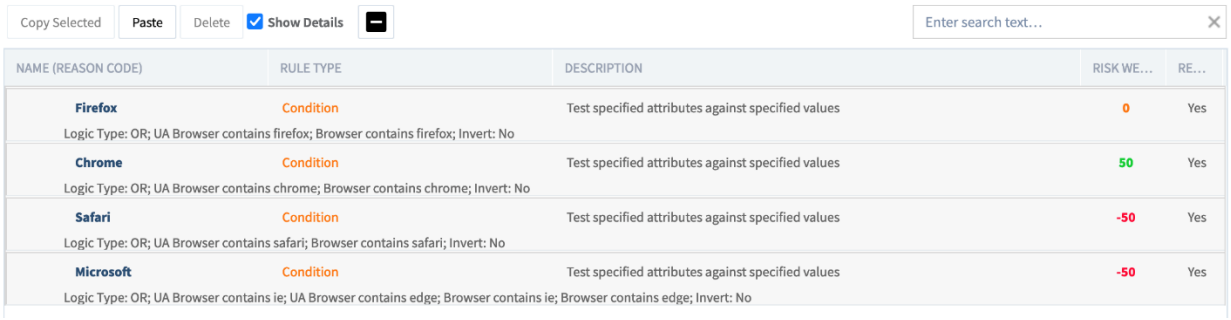
1. Access **DDP Portal** over the internet by logging into your administrative account with credentials provided by LexisNexis Risk Solutions.
2. From the **DDP Portal** home page, select **Policies** from the menu bar. This will provide a listing of available policies that can be configured within the LexisNexis DDP Query node. If a new policy is desired to be created, the instructions here assume the name is **default**. The first step is to select the **Create** dropdown menu followed by **New Policy (Standard)**.



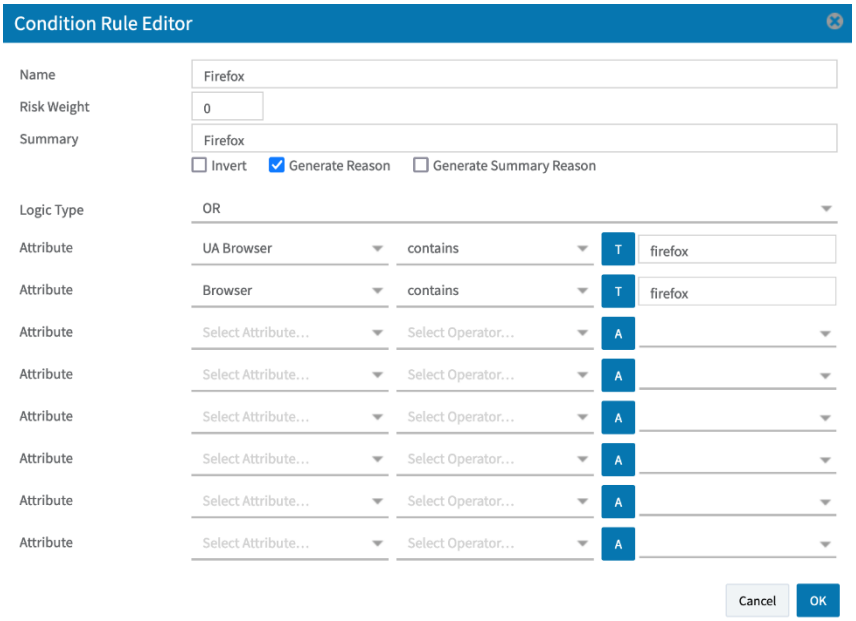
3. On the Policy Summary, the **Properties** interface tab will be displayed. Enter Policy Name = default, select the Active button, and update the Status Thresholds for Reject = -20 and Review = 20.



4. To create the policy rules, select the **Rules** interface tab. The default policy will be a series of **Condition** rules to detect the user browser agent and fulfill the risk weights as follows.



5. Each individual rule can follow the template shown here.



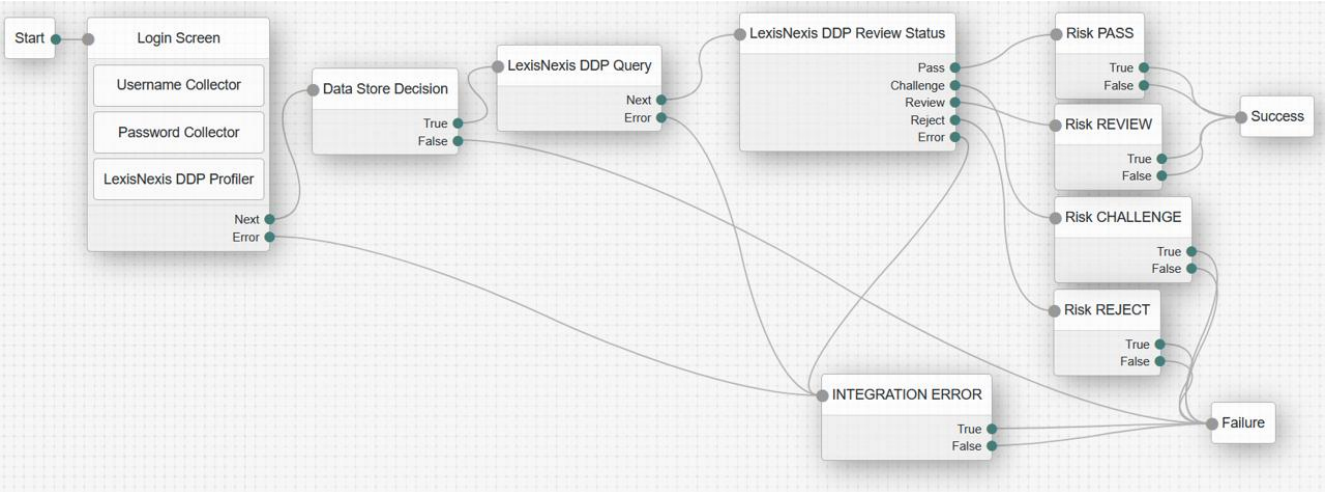
6. Save the policy.
7. Consult with LexisNexis professional services for a more comprehensive policy configuration.

   Another simple test policy configuration can focus on Condition rules that interpret the Account Email attribute for test accounts.

AUTHENTICATION TREE CONFIGURATION

Authentication Tree: Login-Risk-Assessment

This section provides the steps to configure a PingAM / ForgeRock Authentication Tree with LexisNexis DDP nodes from the marketplace. This section will create an authentication tree that performs device intelligence via the LexisNexis DDP Profiler Node followed by a risk assessment via the LexisNexis DDP Query Node. The objective is to have a tree as depicted below.



The flow is as follows:

- Page Node to display the Login user interface that will also contain the DDP Profiling service
- Data Store decision, which is to validate the users credentials against the Identity Store directory
- Risk assessment via the DDP Query service
- Authentication Tree decision using the DDP Review Status node that inspect the risk assessment output of the DDP Query node
- Message Nodes that will display the result of the risk assessment

The steps here will establish the Authentication Tree to be used for authentication of the user. Perform the following to configure the server:

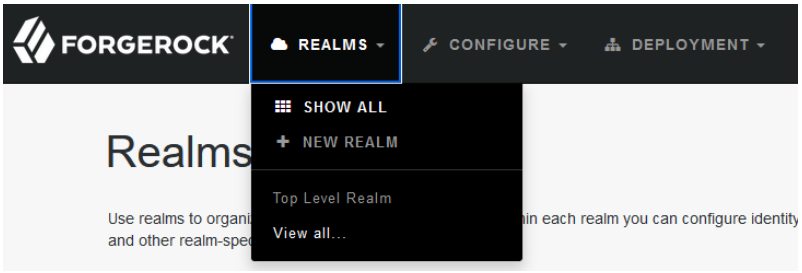
1. From a workstation, launch a browser and enter the URL for the Administration Console:

   Example: `https://sso.threatmetrix.com:8443/openam`

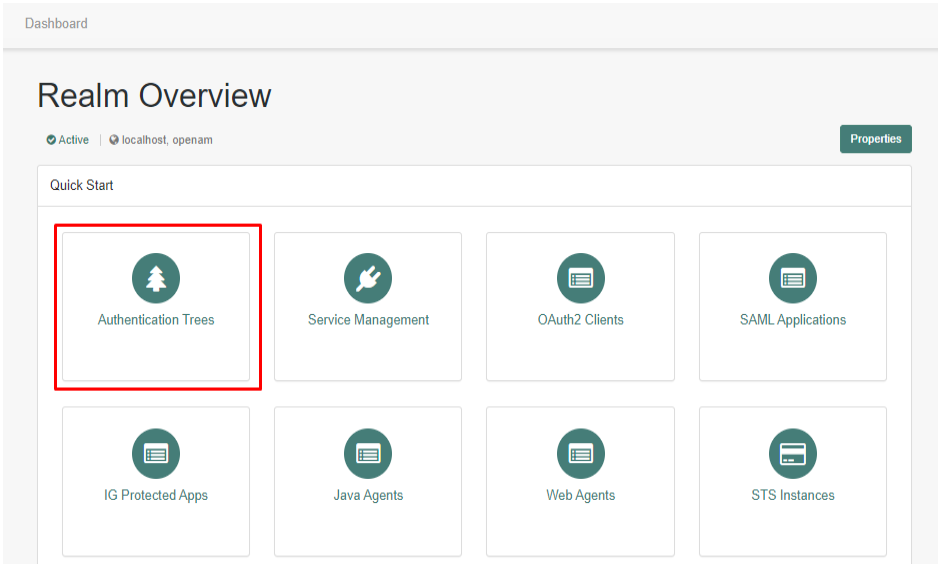
   > **NOTE:** In this example, communications proceed over the HTTPS protocol to a FQDN (`sso.threatmetrix.com`), over a standard Java web container port number (8443), to a specific deployment URI (/openam).

2. Login as Administrator, for example amadmin and credentials

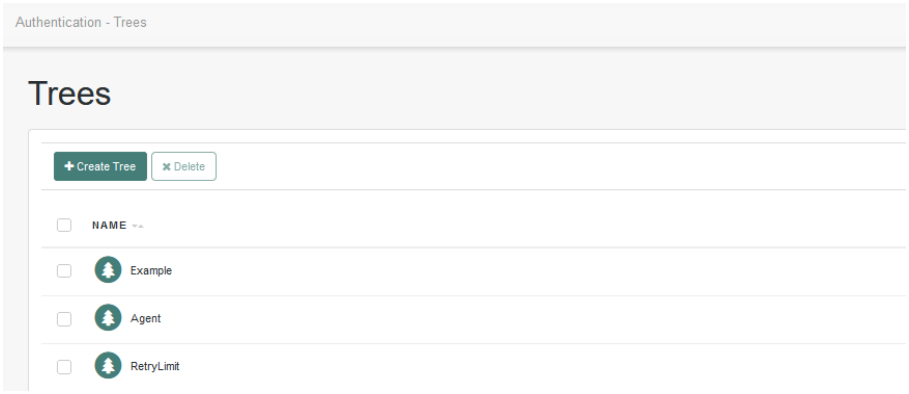
3. Upon login, select the **Realms** dropdown menu and click **Top Level Realm**. For the remainder of this simple quick start configuration, the top-level root realm is assumed.



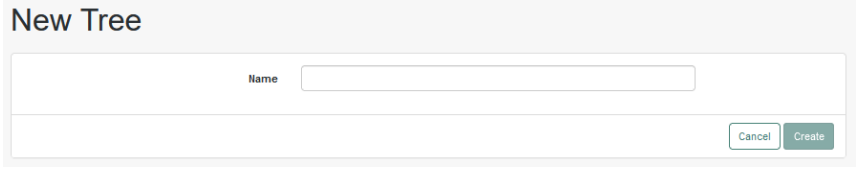
4. On the **Realm Overview** display, click the **Authentication Trees** tile.



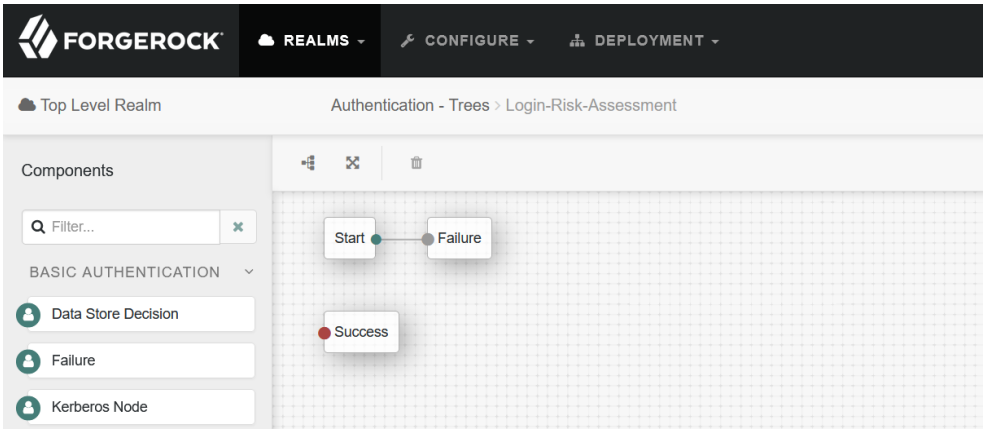
5. On the **Authentication Trees** display, click the **Create Tree** tile.



6. On the **New Tree** display, enter “Login-Risk-Assessment” followed by the **Create** button.



7. The result is the **Authentication Trees > Login-Risk-Assessment** display. This is the interface to build up the authentication policy as a tree depiction showing the nodes in the policy. At this point, the tree will be built by drag-n-drop of Components on the left side of the screen. Each node in the policy will then be configured.

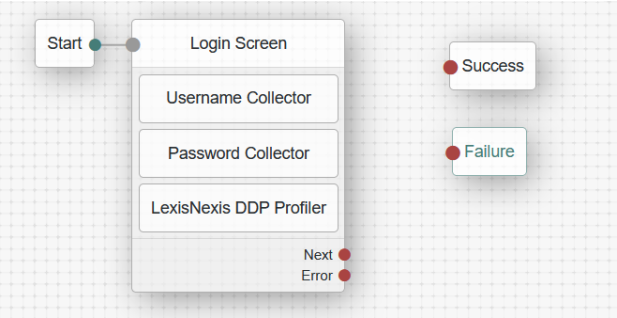


8. Build the Login Screen (e.g. Page Node), do the following:

- On the **Components Filter** on the left side of the interface, enter **page**. When the **Page Node** is displayed as a component, drag and drop the node into the authentication tree.
- When the **Page Node** properties are displayed on the right side of the interface, enter **Login Screen** as the **Node Name**.
- On the **Components Filter** on the left side of the interface, enter **username**. When the **Username Collector** is displayed as a component, drag and drop the **Username Collector** into the authentication tree into the **Login Screen** page node.
- On the **Components Filter** on the left side of the interface, enter **password**. When the **Password Collector** is displayed as a component, drag and drop the **Password Collector** into the authentication tree into the **Login Screen** page node.
- On the **Components Filter** on the left side of the interface, enter **lexisnexis**. When the **LexisNexis DDP Profiler** is displayed as a component, drag and drop it into the authentication tree into the **Login Screen** page node.
- Select the LexisNexis DDP Profiler node component to display the configuration properties on the right side of the interface. Enter the following property values.

Node name	LexisNexis DDP Profiler
Org ID	<ENTER ORG ID FROM DDP PORTAL>
Page ID	Leave Blank – This is optional parameter
Profiler URI	https://h.online-metrix.net/fp/tags.js
Client generated Session ID	Off
Tag Location	HEAD

- At this point you should have the following



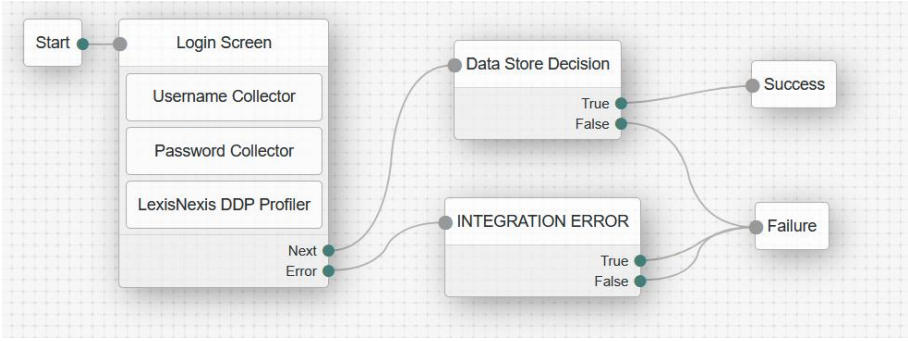
9. Build the Data Store Decision (e.g. login user credential validation), do the following:
   - On the **Components Filter** on the left side of the interface, enter **data**. When the **Data Store Decision** is displayed as a component, drag and drop it into the authentication tree. The data decisions for user credential binding will utilize the Identity Store configuration. User identities are created and managed via the Identities capability.
10. Build Message Node for Integration Error to support testing, do the following:
    - On the **Components Filter** on the left side of the interface, enter **message**. When the **Message Node** is displayed as a component, drag and drop an instance into the authentication tree. This node will be used to display an integration error message from any of the nodes. Enter the following property values.

Node name	INTEGRATION ERROR
Message	(en_US) An integration error has occurred in the Auth Tree
Positive answer	(en_US) OK
Negative answer	(en_US) OK

11. Link together the nodes of the authentication policy
    - To connect the nodes of the authentication tree, click on the output dot of one node and then drag it to the input dot of another node. The following connections should be made.

Start	Login Screen
Login Screen (Next)	Data Store Decision
Login Screen (Error)	INTEGRATION ERROR
Data Store Decision (True)	Success
Data Store Decision (False)	Failure
INTEGRATION ERROR (True)	Failure
INTEGRATION ERROR (False)	Failure

- At this point you should have the following



> **NOTE:** At this point, the authentication tree policy can be tested to validate the login screen and core authentication via Data Store Decision. Using an incognito browser window, enter the following URL and test with the demo user as created in **Identities**.
> URL: https://<DOMAIN>:<PORT>/<ADMIN-URI>/XUI/?realm=/&service=Login-Risk-Assessment
> EX: https://sso.threatmetrix.com:8443/openam/XUI/?realm=/&service= Login-Risk-Assessment
> This URL can be saved as a bookmark as it will be used to test the authentication tree repeatedly.

12. Build the LexisNexis DDP Query Node, do the following:

- On the **Components Filter** on the left side of the interface, enter **lexisnexis**. When the **LexisNexis DDP Query** is displayed as a component, drag and drop it into the authentication tree. This node will get the SessionID calculated by the **LexisNexis DDP Profiler** to perform the session_query API for risk assessment.
- Select the **LexisNexis DDP Query Node** component to display the configuration properties on the right side of the interface. Enter the following property values.

Node name	LexisNexis DDP Query
Org ID	<ENTER ORG ID FROM DDP PORTAL>
API Key	<ENTER API KEY FROM DDP PORTAL>
Service Type	Session Policy
Event Type	Login
Policy	default
Unknown Session Action	Challenge (This allows knowing if condition is encountered)
Query Type	Session Query
Base URL	https://h-api.online-metrix.net/
Add Attributes to API Request	Selected
Attribute Source	User Directory
Query Attributes	Key=account_email, Value=mail Key=account_last_name, Value=sn Key=account_first_name, Value=givenName

> **NOTE:** The Query Attributes will attempt to discover the values as configured by the Attribute Source. This is a list of DDP attributes (e.g. "key") to authentication journey attributes (e.g. "value"). If the values cannot be fetched, the API Request will not include the DDP attribute.

NOTE: The Attribute Source defines where the Query Attributes will be searched. User Directory will look for attributes in the Identity Store, and Shared State looks in the shared memory of the authentication tree.

NOTE: The Query Type configuration allows the integrator to define if the API includes device profiling (e.g. Session Query) or the API is basic (e.g. Attribute Query). In the case of the Session Query configuration, the LexisNexis DDP Profiler Node is required that will create the Session ID.

NOTE: The Unknown Session Action configuration allows the administrator to define an outcome when LexisNexis DDP Profiling fails to process correctly. Having this outcome configuration can avoid users from being denied access based on issues with the DDP Service. For the purposes of this instruction, the values are set the "Challenge".

13. Build the LexisNexis DDP Review Status Node, do the following:

- On the **Components Filter** on the left side of the interface, enter **lexisnexis**. When the **LexisNexis DDP Review Status** is displayed as a component, drag and drop it into the authentication tree. This node will get the result of risk assessment from the **LexisNexis DDP Query** to perform decision logic on how to branch based on the review_status attribute in the API Response.
- Select the **LexisNexis DDP Review Status** component to display the configuration properties on the right side of the interface. Enter the following property values.

Node name	LexisNexis DDP Review Status

14. Build Message Nodes for the outcomes to support testing, do the following:

- On the **Components Filter** on the left side of the interface, enter **message**. When the **Message Node** is displayed as a component, drag and drop four instances into the authentication tree. These nodes will be used to display the outcome of the **LexisNexis DDP Review Status**.
- Name the four nodes as follows:

Node name	Risk PASS
Node name	Risk CHALLENGE
Node name	Risk REVIEW
Node name	Risk REJECT

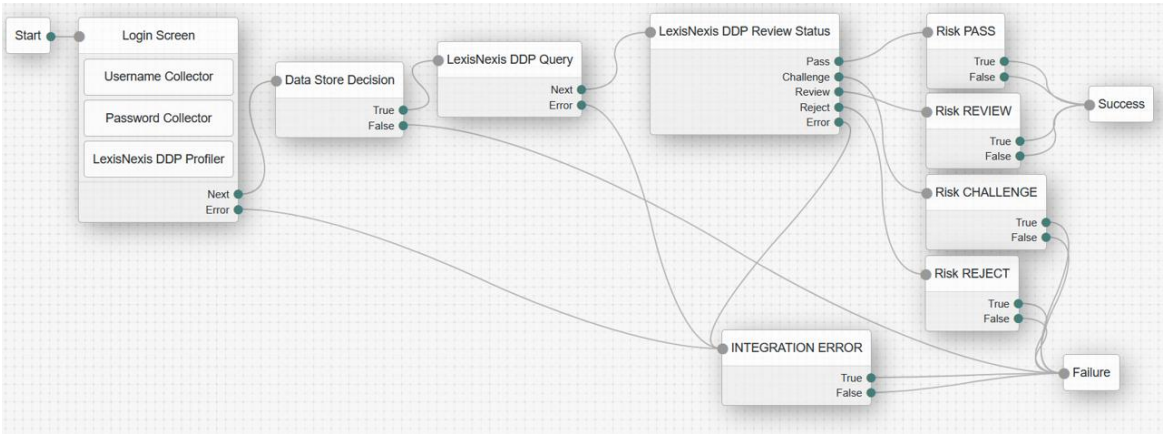
15. Link together the nodes of the authentication policy

- To connect the nodes of the authentication tree, click on the output dot of one node and then drag it to the input dot of another node. The following connections should be made.

Start	Login Screen
Login Screen (Next)	Data Store Decision
Login Screen (Error)	INTEGRATION ERROR
Data Store Decision (True)	LexisNexis DDP Query
Data Store Decision (False)	Failure
INTEGRATION ERROR (True)	Failure
INTEGRATION ERROR (False)	Failure
LexisNexis DDP Query (Next)	LexisNexis DDP Review Status

LexisNexis DDP Query (Error)	INTEGRATION ERROR
LexisNexis DDP Review Status (Pass)	Risk PASS
LexisNexis DDP Review Status (Challenge)	Risk CHALLENGE
LexisNexis DDP Review Status (Review)	Risk REVIEW
LexisNexis DDP Review Status (Reject)	Risk REJECT
LexisNexis DDP Review Status (Error)	INTEGRATION ERROR
Risk PASS (True)	Success
Risk PASS (False)	Success
Risk Assessment REVIEW (True)	Success
Risk Assessment REVIEW (False)	Success
Risk Assessment CHALLENGE (True)	Failure
Risk Assessment CHALLENGE (False)	Failure
Risk Assessment REJECT (True)	Failure
Risk Assessment REJECT (False)	Failure

- At this point you should have the following



16. The risk assessment Authentication Tree with LexisNexis DDP nodes is complete and ready for testing. In order to test, ensure a test user is configured via **Identities**. The configured tree will utilize the **Identity Store** to authenticate the user and pull any additional query attributes.
    - URL: https://<DOMAIN>:<PORT>/<ADMIN-URI>/XUI/?realm=/&service=Login-Risk-Assessment
    - EX: https://sso.threatmetrix.com:8443/openam/XUI/?realm=/&service= Login-Risk-Assessment

Authentication Tree: LNRS-StepUp-Update

This configuration in this section of the quick start guide is optional. This section provides the steps to configure an Authentication Tree with LexisNexis DDP nodes from the marketplace, specifically how to provide truth data and final review status. This integration is performed when the review status indicates a step-up authentication is needed based on the risk assessment. For this simple example, a Message Node will be used to simulate One-Time Passcode (OTP).

This section will create an Authentication Tree that is meant to be called from an Inner Tree Evaluator node. The objective is to package the functionality of the step-up authentication into a separate Authentication Tree that is then invoked from the primary risk assessment Authentication Tree, which in this case is the Login-Risk-Assessment authentication tree from the previous section.

Perform the following to configure:

1. From a workstation, launch a browser and navigate to the Access Management Admin Console.
2. Login with amadmin and credentials
3. Upon login, select the **Realms** dropdown menu and click **Top Level Realm**.
4. On the **Realm Overview** display, click the **Authentication Trees** tile.
5. On the **Authentication Trees** display, click the **Create Tree** tile.
6. On the **New Tree** display, enter "LNRS-StepUp-Update" followed by the **Create** button.
7. The result is the **Authentication Trees > LNRS-StepUp-Update** display. At this point, the tree will be built by drag-n-drop of Components on the left side of the screen. Each node in the policy will then be configured.
8. Build the **LexisNexis DDP Update** nodes, do the following:
   - There are going to be three (3) nodes in the tree to handle the different outcomes from the review status node. So be sure to have an "init", "pass" and "fail" node.
   - On the **Components Filter** on the left side of the interface, enter **lexisnexis**. When the **LexisNexis DDP Update Review** is displayed as a component, drag and drop it into the authentication tree. This node will add retrospective truth data to a DDP query event that indicates step-up authentication is being initialized. Once the node has been placed on the Authentication Tree graph, select the component and enter the following configuration:

Node name	Update Review INIT
Org ID	<ENTER ORG ID FROM DDP PORTAL>
API Key	<ENTER API KEY FROM DDP PORTAL>
Base URL	https://h-api.online-metrix.net/
Event Tag	Step-Up Initialize
Step-Up Method	OTP SMS – This can be set to type of OTP used for MFA
Notes	<BLANK> - This is an optional parameter

   - On the **Components Filter** on the left side of the interface, enter **lexisnexis**. When the **LexisNexis DDP Update Review** is displayed as a component, drag and drop it into the authentication tree. This node will add retrospective truth data to a Session Query event that indicates step-up authentication is being initialized. Once the node has been placed on the Auth Tree graph, select the component and enter the following configuration:

Node name	Update Review PASS
Org ID	<ENTER ORG ID FROM DDP PORTAL>
API Key	<ENTER API KEY FROM DDP PORTAL>
Base URL	https://h-api.online-metrix.net/
Event Tag	Step-Up Pass
Step-Up Method	OTP SMS – This can be set to type of OTP used for MFA
Notes	<BLANK> - This is an optional parameter

- On the **Components Filter** on the left side of the interface, enter **lexisnexis**. When the **LexisNexis DDP Update Review** is displayed as a component, drag and drop it into the authentication tree. This node will add retrospective truth data to a Session Query event that indicates step-up authentication is being initialized. Once the node has been placed on the Auth Tree graph, select the component and enter the following configuration:

Node name	Update Review FAIL
Org ID	<ENTER ORG ID FROM DDP PORTAL>
API Key	<ENTER API KEY FROM DDP PORTAL>
Base URL	https://h-api.online-metrix.net/
Event Tag	Step-Up Fail
Step-Up Method	OTP SMS – This can be set to type of OTP used for MFA
Notes	<BLANK> - This is an optional parameter

9. Build Message Node for Integration Error to support testing, do the following:

- On the **Components Filter** on the left side of the interface, enter **message**. When the **Message Node** is displayed as a component, drag and drop an instance into the authentication tree. This node will be used to display an integration error message. Enter the following property values.

Node name	INTEGRATION ERROR
Message	(en_US) An integration error has occurred in the Auth Tree
Positive answer	(en_US) OK
Negative answer	(en_US) OK

10. Build Inner Tree Evaluator to invoke an MFA process. For the purposes of this instruction, it is assumed an OTP Authentication Tree for OTP exists. Do the following:

- On the **Components Filter** on the left side of the interface, enter **inner**. When the **Inner Tree Evaluator** is displayed as a component, drag and drop an instance into the authentication tree. This node will simulate the success or failure of Step-Up Authentication. Enter the following property values.

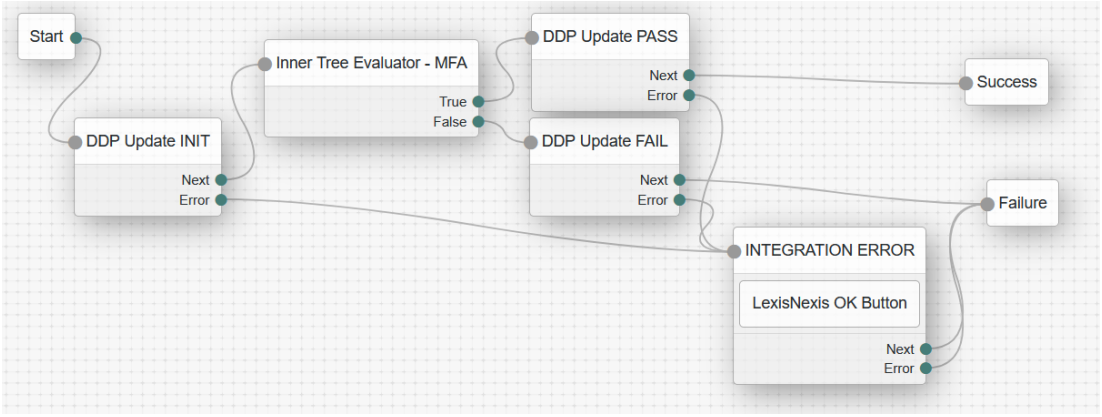
Node name	Inner Tree Evaluator - MFA
Tree Name	<Select appropriate Tree for OTP>

11. Link together the nodes of the authentication policy

- To connect the nodes of the authentication tree, click on the output dot of one node and then drag it to the input dot of another node. The following connections should be made:

Start	Update Review INIT
DDP Update INIT (Next)	Inner Tree Evaluator - MFA
DDP Update INIT (Error)	INTEGRATION ERROR
Inner Tree Evaluator - MFA (True)	DDP Update PASS
Inner Tree Evaluator - MFA (False)	DDP Update FAIL
DDP Update PASS (Next)	Success
DDP Update PASS (Error)	INTEGRATION ERROR
DDP Update FAIL (Next)	Failure
DDP Update FAIL (Error)	INTEGRATION ERROR
INTEGRATION ERROR (True)	Failure
INTEGRATION ERROR (False)	Failure

- At this point you should have the following



12. At this point the authentication tree to update truth data for a DDP Risk Assessment is ready to be called from any other authentication tree. Click **Save**.

13. To leverage the update truth data from the Login Risk Assessment authentication tree, add an Inner Tree Evaluator node.

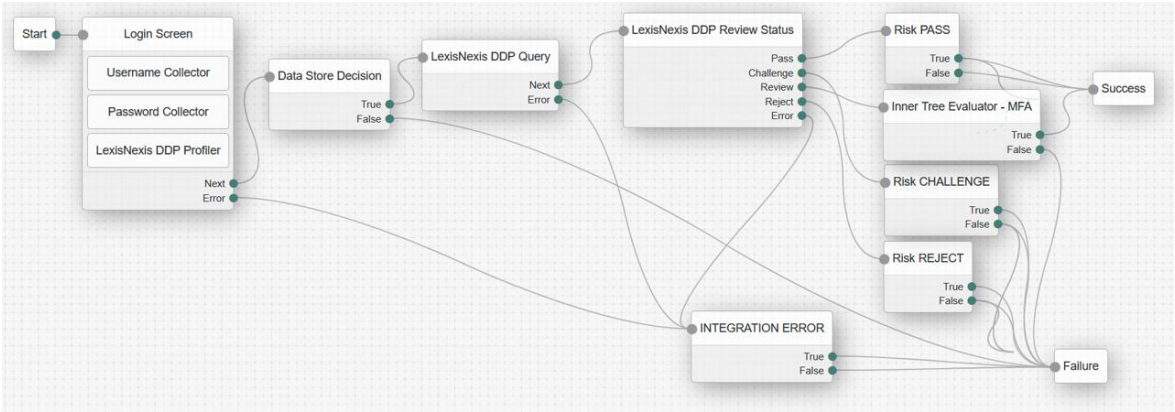
    - Open the Authentication Tree named Login-Risk-Assessment
    - On the **Components Filter** on the left side of the interface, enter **inner**. When the **Inner Tree Evaluator** node is displayed as a component, drag and drop an instance into the authentication tree. This node will be used to invoke the OTP Step Up authentication tree. Enter the following property values.

Node name	Inner Tree Evaluator - MFA
Tree Name	LNRS-StepUp-Update

    - To connect the nodes of the authentication tree, click on the output dot of one node and then drag it to the input dot of another node. The following connections should be made and/or updated:

LexisNexis DDP Review Status (Review)	Inner Tree Evaluator - MFA
Inner Tree - OTP StepUp (True)	Update Review PASS
Inner Tree - OTP StepUp (False)	Update Review FAIL

    - At this point you should have the following



14. The risk assessment and step-up truth data Authentication Trees with LexisNexis DDP nodes are complete and ready for testing. In order to test, ensure a test user is configured via **Identities**. The configured tree will utilize the **Identity Store** to authenticate the user and pull any additional query attributes.
    - URL: https://<DOMAIN>:<PORT>/<ADMIN-URI>/XUI/?realm=/&service=Login-Risk-Assessment
    - EX: https://sso.threatmetrix.com:8443/openam/XUI/?realm=/&service= Login-Risk-Assessment